Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222 Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	26-27	Semester:	3-1
Name of the Student:	B.Bhavisyavani	Roll Number:	A24126552079
Name of the Lab:	FS lab	Timings:	9:40 to 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
1/7	1	3	4	5	1	13		
8/7	2	3	4	5	1	13		
15/7	3	4	4	5	1	14		
22/7	4	3	4	5	1	13		
29/7	5	4	3	5	2	14		
5/8	6	4	3	5	1	13		
12/8	7	4	3	5	2	14		
19/8	8	4	4	5	2	15		
2/9	9	4	3	5	2	14		
9/9	10	4	4	5	2	15		

Name of the Faculty: Ch. Sruthi

Faculty Signature: Sruthi

Introduction to HTML and CSS?

→ HTML (Hyper text Markup language) is the standard Markup language used to create and structure content on web pages. it defines the basic structure of a webpage using elements and tags such as headings paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage,

→ CSS ! (Cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents it is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be

applied using inline, internal, or external Stylesheets.

→ HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. the combination of both forms the foundation of modern web development and enables the creation of organized attractive, and user-friendly web interfaces.

Difference between MERN stack & FULL stack>

→ MERN stack is a JavaScript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework, and APIs, React.js is used to develop frontend user interface, and Node.js provides the server-side runtime environment.

→ Full-stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used designing the user interface, implementing server-side logic, managing databases.

HTML Components:

1. HTML Tags:- Define the structure and elements of a webpage.
2. Headings:- Used to display titles and headings using <h1> to <h6>
3. Paragraphs:- Used to display text using the <p> tag.
4. Links:- Used to navigate between webpages using <a> tag.
5. Images:- Used to display images using <image> tag.
6. Lists:- Used to create ordered and unordered lists using <ol>, <ul>, <li>.

7. Tables:- Used to organize data in rows and columns.

8. Forms:- Used to collect user input using elements such as textboxes, buttons and checkboxes.

9. Div and span:- Used to group and organize webpage content.

10. Semantic Elements:- Elements such as <header>, <nav>, <section>, <article>, <footer> provide meaningful webpage structure.

CSS Components:-

1. selectors:- Select HTML elements to apply styles.

2. properties:- Defines the visual characteristic of elements.

3. Values:- Specify the setting of a css property.

4. colors:- Defines text, background, border colors.

5. Fonts:- Control font family, size, style and weight

6. Backgrounds:- Set background color and images.

7. Borders:- Define the border style, width and radius.

8. Margins:- Controls the space outside an element.

9. Padding:- Controls the space inside element.

10. Layout:- Controls the arrangement of elements using Flexbox, Grid and positioning.

HTML VS HTML5

HTML	HTML5
→ Older version of HTML	→ Latest Major version HTML
→ Limited multimedia support	→ Supports audio and video.
→ Uses External plugins for multimedia.	→ No external plugs needed.
→ Has fewer semantic ~~fewer~~ tags	→ Provides semantic tags
→ No <canvas> element	→ Support <canvas>
→ Limited web storage	→ Support local storage
→ Complex DOCTYPE declaration	→ Simple <! DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
→ Earlier version of CSS	→ Advanced version of CSS
→ Basic styling features	→ Provides advanced styling features.
→ Limited Animations	→ Supports animation and transition.
→ No media queries	→ Supports media queries.
→ Uses a single specification	→ Divided into different modules

15/07/26

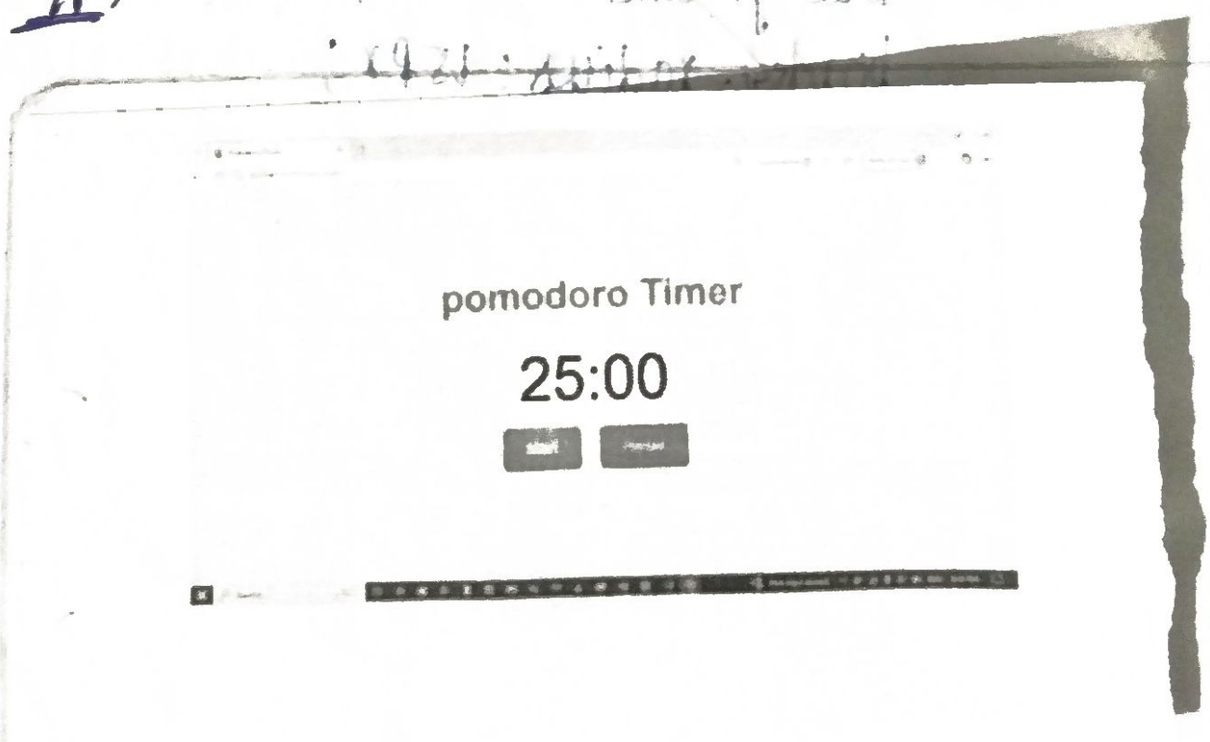
Design a static webpage using HTML and CSS.

Pomodoro timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
     body {
         font-family : Arial;
         background : #f5f5f5;
         text-align : center;
         padding : 50px;
        }
       .timer {
            width : 300px;
            margin : auto;
            padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }
      h1 { color : #e74c3c;
         }
       .time {
            font-size : 50px;
            margin : 5px;
        }
```

```
button {
    Padding : 10 Px  20px;
    margin: 5px;
    border : none;
    border-radius : 5px;
    background : #e74c3c;
    color: white;
}
</style>
</head>
<body>
    <div class = "Timer">
    <h1> Pomodoro Timer </h1>
    <div class = "Time"> 25:00 </div>
    <button> Start </button>
    <button> Reset </button>
</div>
</body>
</html>
```

O/P :



22/07/2026.

1) Design an HTML code using audio tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favorite song </h2>
    <audio controls>
        <source src="song.mp3"
            type="audio/mpeg">
        Your browser does not support the
    audio tag
    </audio>
</body>
</html>
```

O/p: displays an audio player with play, pause and volume controls

Steps:

Create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with control attributes
Specify the audio file using <source> tag.
Save and open the HTML file in web browser.

1i) Design a HTML Code Using Video tags

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Exampl
</head>
<body>
    <h2> My Video </h2>
    <video width = "400"
        <source src = "vide
                type = "
    </video>
</body>
</html>
```



O/P: Displays a video player with play, Pause, volume, and full screen controls.

Steps:

Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag.
Insert the <video> tag with control attributes
Specify the video file using <source> tag.
Save the file and open it in web browser
Play the video using displayed controls.

3 Write a program using Javascript (arrays, functions)

Code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray (arr) {
    console.log ("Array Elements:");
    for (let i=0; i<arr.length; i++)
  { console.log(arr[i]);
 }
}
function findSum (arr) {
    let sum =0;
    for (let i=0; i< arr.length; i++) {
      sum+ = arr[i];
  }
    return sum;
}
```

O/P: Array elements : 12
45
8
23
56
19
34
67
90
11

Sum = 365

Sum of Array :

```
function MaxArray(arr) {
    let Max = arr[0];
    for (let i=0; i<arr.length; i++)
    if (arr[i] > Max) {
        Max = arr[i];
    }
}
```

O/P :- 90

Find Minimum number in Array

```
function findMin(arr) {
    let Min = arr[0];
    for (let i=0; i<arr.length; i++)
    {
        if (arr[i] < Min) {
            Min = arr[i];
        }
    }
    return Min;
}
Console.log("Minimum Number:",
            findMin(Number));
```

O/P :- 8

print reverse array

```
function rev(arr)
{
    return arr.reverse();
}
let fruits = ['apple', 'banana', 'apricot',
              'orange', 'grape'];
console.log("Reverse fruits:", rev(fruits));
```

O/P :- ['grape', 'orange', 'apricot', 'banana', 'apple'].

4. Develop Dynamic webpage using Javascript

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Student Management </title>
<style>
    body {
        font-family: Arial, Sans-serif;
        background-color: #f2f2f2;
        Margin: 0;
    }
 header {
        background-color: #333;
         color: white;
         text-align: center;
         Padding: 20px;
    }
 .container {
         width: 80%;
         Margin: 30px auto;
         background-color: white;
         Padding: 25px;
         border_radius: 10px;
      }
input {
        Padding: 10px;
        Margin: 5px;
        width: 200px;
  }
button {
      Padding: 10px 20px;
       Margin: 5px;
       border: none;
       background-color: #333;
       color: white;
       cursor: pointer;
       border-radius: 5px;
   }
 button:hover {
        background-color: #555;
  }
table {
      width: 100%;
      Margin-top: 20px;
      border-collapse: collapse;
  }
th, td {
      border: 1px solid #ccc;
      Padding: 12px;
      text-align: center;
  }
th {
    background-color: #333;
     color: white;
  }
.delete {
      background-color: red;
    }
.stats {
     Margin-top: 20px;
     font-size: 18px;
   font-weight: bold;
```

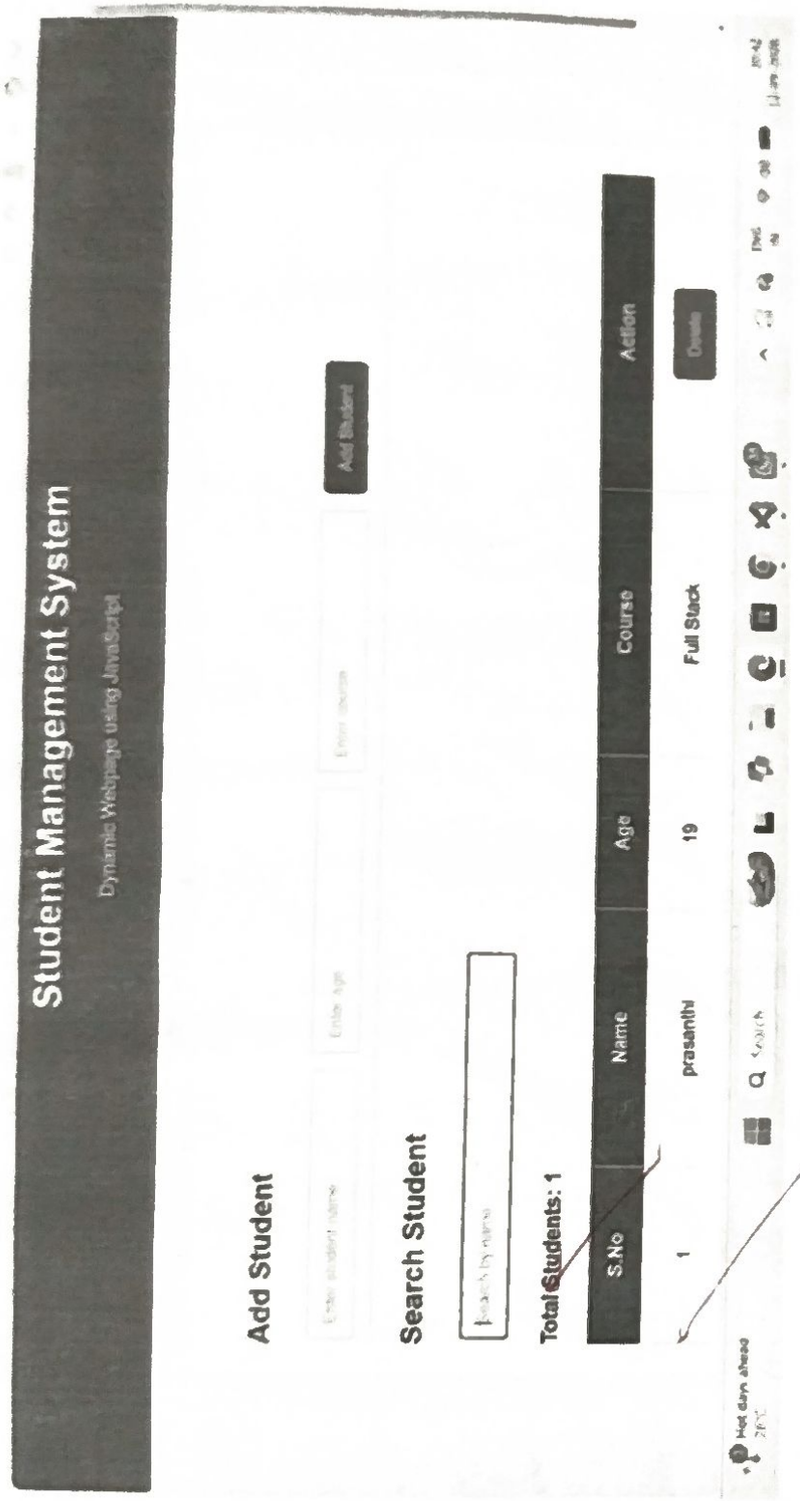
```
#search {
        width : 300px;
    }
</style>
</head>
<body>
<header> <h1> Student Management System </h1>
        <p> Dynamic webpage using Javascript </p>
</header>
<div class = "container">
    <h2> Add Student </h2>
  <input type = "text" id = "name"
      Placeholder = "enter student name">
  <input type = "number" id = "age"
        Placeholder = " Enter age">
<input type = "text" id = "course"
        placeholder = "enter Course">
button onclick = "addstudent()"> Add student
                                </button>
br>
h2> search student </h2>
input type = "text"
      id = "search"
      Placeholder = "search by name"
      onkeyup = " SearchStudent()">
<div class = "status">
```

```
Total Students: <span id = "count"> 0 </span>
</div>
<table>
   <thead>
    <tr>
     <th> S.NO </th>
     <th> Name </th>
     <th> Age </th>
     <th> Course </th>
     <th> Action </th>
  </tr>
 </thead>
 <tbody id = " student table">
 </tbody>
 </table>
</div>
<script>
let Students = [];
function add Student() {
    let name = document.getElementById("name").
                                    value;
 let age = document.getElementById ("age").
                                    value;
let course = document.getElementById ("course").
                                    value;
if (name === " " || age === " " || course === "")
{
    alert ("Please enter all details!");
    return;
}
```

```
let Student = {
  name: name,
  age : age,
  course: course
};
Student.push (student);
displayStudents ();

document.getElementById ("name").value = "";
document.getElementById("age").value = "";
document.getElementById ("course").value = "";
}
function displayStudents() {
    let table =document.getElementById ("Student table");
    table.innerHtml = "";
  Students. ForEach (function (Student, index) {
  let row =
  <tr>
     <td> ${index+1} </td>
     <td> ${Student.name} </td>
     <td> ${student.age} </td>
     <td> ${student. course} </td>
     <td>
  <button  class = "delete"
     onclick = "deleteStudent (${index})" >
     Delete  </button>
</td>
</tr> ;
table.innerHTML += row;
});
```

```
document.getElementById ("count").innerText =
                                  Students.length;
}
 function deleteStudent (index) {
     Student. splice (index,1);
  } display Students();
  function SearchStudent() {
 let searchValue =
          document.getElementId ("search").value.
                                      toLowerCase();
let rows =
 document.getElementById ("student table").get
  ElementByTagName ("tr");
  for (let i=0; i<rows.length; i++) {
let name = rows[i].getElementByTagName ("id")
                                          [1];
if (name) {
   let StudentName =
          name.textContent.toLowerCase();
  if (StudentName.includes (searchvalue)) {
     rows[i].style.display = "";
  }
  else {
     rows[i].style.display = "none";
}
  }
  }
</script>
</body>
</html>
```



5) Implement Modules in Node.js

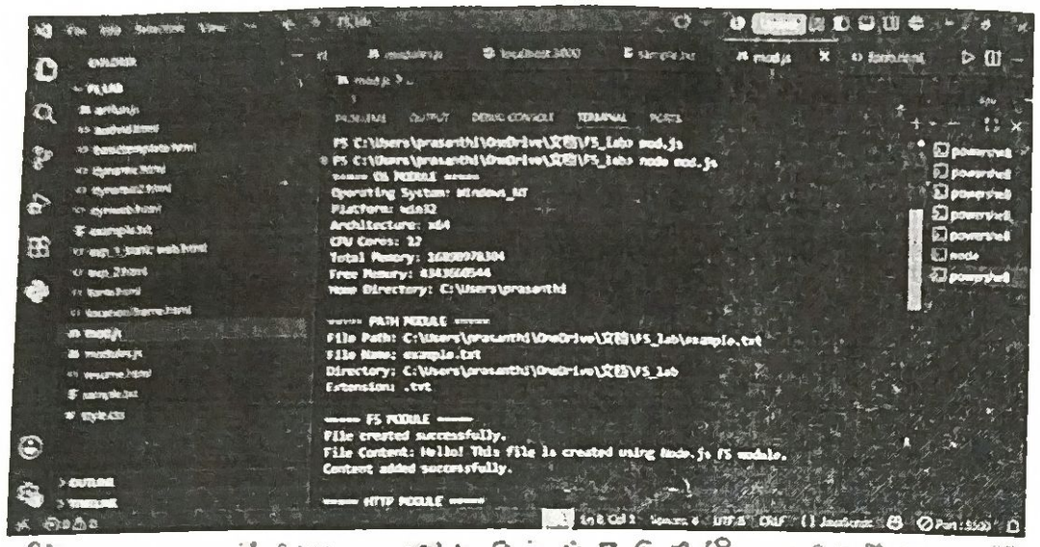
Modules.js

```
Const os = require("os");
Const fs = require ("fs");
Const path = require ("path");
Const http = require ("http");
Const net = require ("net");
Console.log ("===os, Module ===");
Console.log ("operating system :", os.type());
Console.log ("platform :", os.platform());
Console.log ("Attrib Architecture :", os.arch());
Console.log ("cpu cores :", os.cpus().length);
Console.log ("Total memory:", os. totalmem());
Console.log ("Free memory:", os: freemem());
Console.log ("Home Directory :", os.homedir());
Console.log ("\n === PATH MODULE ===");
Const filepath = path.join (dirname, "example.txt");
Console.log ("File path :", filepath);
Console.log ("File Name:", path.basename (filepath));
Console.log ("Directory :", path.dirname (filepath));
Console.log ("extension:", path.extname (filepath));
Console.log ("\n === FS MODULE ===");
fs.writeFileSync (
                filepaths,
```

```
"Hello ! This file is created using Node.js
    fs Module : ");
Console.log ("File Created Successfully ");
Const data = fs. readFileSync (Filepath, "utf-8 ");
Console.log ("File Content :", data);
fs.appendFile Sync (
        File path,
    "\n This is additional Content ");
Console.log ("content added successfully ");
Console.log ("\n = = = HTTP MODULE = = = ");
Const Server = http.(createServer (req, res) =>)
{ res. writeHead (200, {
    "Content-Type" : "text/html "
});
res.write ("<h1> Hello from Node-js.</h1>");
res.write ("<p> this Server is Created using
    HTTP Module .</p> ");
res.end();
});
Server.listen (8000, () => {
Console.log ("HTTP Server running at .
  http:// localhost : 8000 ");
});
```

```
Console.log ("\n = = = NET MODULE
                - = - ");
Const netserver = net. (createServer (socket) =>
{
  Console.log ("client connected. ");
  Socket.write (
    "Hello Client! Welcome to Node.js
     NET Server. "
  );
Socket.on ("data ", (data) => {
      Console.log ("client says :",
          data.toString ());
});
Socket.on ("end", () => {
      Console.log ("client disconnected ");
});
});
netServer.listen (5000, () =>
  {
    Console.log ("NET Server running on
      Port 5000 ");
  });
```

O/P:




```
Problems  Output  Debug Console  Terminal  Ports

C:\Users\user\Desktop\SAI\NodeModules>cd ..

C:\Users\user\Desktop\SAI>cd student-app

C:\Users\user\Desktop\SAI\student-app>node app.js
Server running at http://localhost:3000
```

```
C:\Users\user\Desktop\SAI>cd NodeModules

C:\Users\user\Desktop\SAI\NodeModules>node app.js
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2

C:\Users\user\Desktop\SAI\NodeModules>
```

6) Develop Mini application using Express and Node.js.

Server.js :-

```
Const express = require ("express");
const path = require ("path");
Const app = express();
const port = 2000;
app.use(express.json());
app.use(express.urlencoded({ Extended: true }));
app.use(express.static(dirname));
app.get("/", (req, res) => {
    res.sendFile(path.join(dirname,
        "index.html"));
});
app.post("/login", (req, res) => {
  Const username = req.body ? res.body.username:
                                undefined;
Const password = req.body ? req.body.
                        Password: undefined;
if (!username || !password) {
      return res.status(400).json({
        Success: false,
        Message: "Please enter username
                  and Password"
    });
}
```

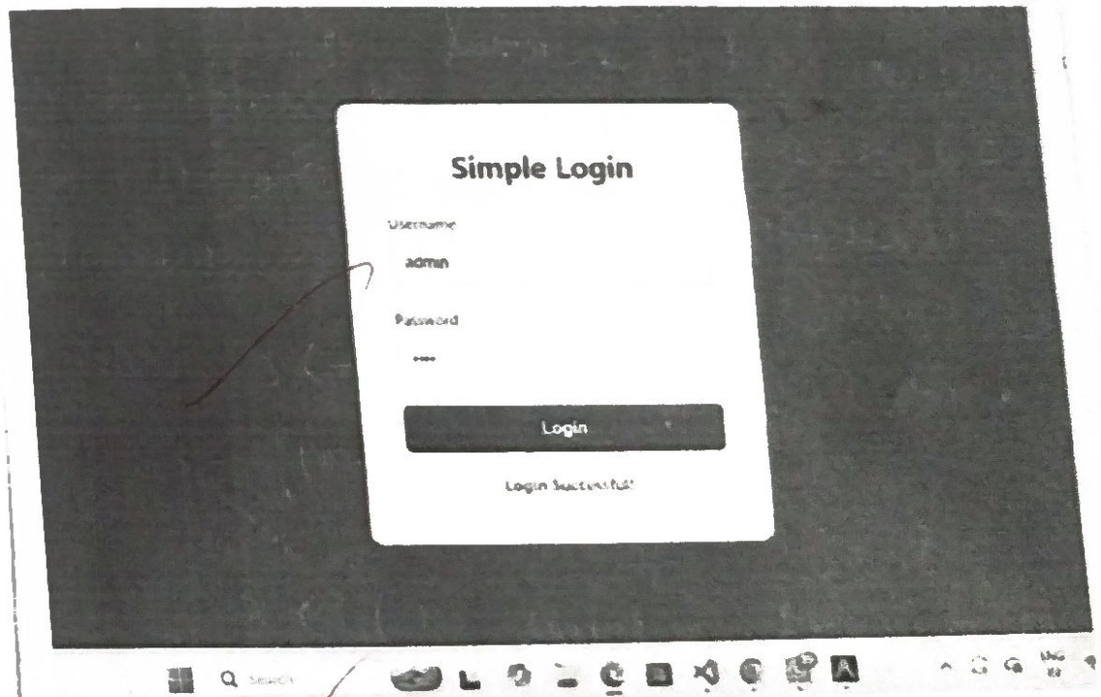
```
if (username === "admin" && password === "1234") {
  return res.json({
    success: true,
    message: "Login Successfull!"
  });
} else {
  return res.status(401).json({
    success: false,
    message: "Invalid username or password"
  });
}
});

app.listen(PORT, () => {
  console.log("Server running at http://
  localhost:${PORT}");
});
```

Index.html :-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title> Simple login </title>
  </head>
  <body>
   <div class="login-card">
    <h1> Simple login </h1>
    <form onsubmit="event.preventDefault();
        login();">
    <div class="form-group">
     <label for="username"> username </label>
     <input type="text" id="username"
      placeholder="enter username" required>
    </div>
     <div class="form-group">
      <label for="password"> password
      </label>
     <input type="password" id="password"
      placeholder="enter password" required>
     </div>
     <button type="submit"> login </button>
    </form>
    <h3 id="result"></h3>
    </div>
    <script>
     async function login() {
       const usernameInput = document.getElement-
                    ById("username");
```

```
const passwordInput = document.getElementById
                        ("Password");
const result[] = document.getElementById("result");
const username = UsernameInput.value.trim();
const password = PasswordInput.value.trim();
if (!username || !password)
{
   result El.innerText = "fill fields";
   result El.className = "error";
   return;
}
try {
   const response = await fetch("/login", {
     method : "POST",
     headers : {
        "content-Type" : "application/json"
     },
     body : JSON, stringify ({ username,
                               password })
   });
const data = await. response.json();
result €.inner text = data.message;
if (response.ok && data.success !== false)
  {
    result.className = "success";
}
else {
  result El. className = "error";
}
  catch (err) {
    result El.innerText = "server connection error!";
    result.className = "error";
}
</script>
</body>
</html>
```



7. Create a Functional Component and class Component using react.

Aim: To develop a functional Component and class Component using react.

Function Component:-

- it is a Javascript functional Component or that returns JSX to display that user interface. it uses React Hooks such as Usestate to manage state and update the webpage dynamically.

Program:-

```
import React, { Usestate } from 'react';
function Studentfunctional () {
    const [name, setName] = UseState ('Bhavisya');
    const [course, setcourse] = Usestate ("CSM");
    Const [course, setcount] = UseState(0);
    const changeName = () => {
        setName ("vishya");
    };
    const changecourse = () => {
        setcourse ("CSD");
};
    const IncrementCount = () => {
        setCount (count++)
    };
```

```
return (
    <div style = {
        border: "2px solid blue";
        Padding: "20px",
        Margin: "20px";
        borderRadius: "10px";
        text align: "center"
    }}>
<h1> Student Details </h1>
<h2> functional Component </h2>
<p> <b> Name: </b> {name} </p>
<p> <b> Course: </b> {course} </p>
<p> <b> button clicks: </b> {count} </p>
<button onclick = {change Name}>
        Change Name </button>
<button onclick = {change Name course}
        change course </button>
<button onclick = {Increment Count}>
    Increment </button>
</div>
```

output

when the user clicks the buttons, the name, course and Count dynamically without reloading the web page.

Class component:

A class component is an ES6 javascript class that extends React component. It uses the render() method to return JSX and manages state using this.state and this.setstate().

Program :-

```
import React { Component } from
"react":
class studentclass extends Component {
  constructor (props) {
        super (props);
        this.state = {
          name : "sai",
          course : "Computer science",
          count = 0
    };
  }
changeName = () => {
        this.setstate ( {
              name : "sai"
        });
  };
change course = () => {
      this.setState ( {
            course : "CSM"
      });
  };
```

```
increment count = () => {
      this.setstate (prevstate) => ( {
        count. prevstate.count,
    }));
};
render() {
 return (
      <div style = {
            border : "2px solid green",
            Padding : "20px";
            Margin = "20 px;
            border Radius = "10px;
            textalign : "center"
        }}>
   <h1> Student Details </h1>
    <h2> class Component </h2>
 <p> <b> Name : </b> {this.state.name}
                                </p>
 <p> <b> Course : </b> {this.state.course}
                                </p>
 <p> <b> Button clicks : </b> {this.state.
                                count} </p>
<button onclick = {this.change Name}>
   Change Name </button>
<button onclick = {this.change course}>
   Change course </button>
<button onclick = {this.increment count}>
   <div> Increement </button>
```

React Components

Student Details

Functional Component

Name: Prasanthi

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Prasanthi

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

8. Implement http methods using JSON server.

Aim:- To implement HTTP Methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update and Delete operations on Student records.

Theory:- JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate b/w a client & a server.

Step 1: Initialize the project

- Open the VS Code terminal and execute the following commands:

```
Mk dir JSONserver
Cd JSON server
npm init.4
npm install json server
```

Step 2: Create dbjson

Create a file named db.json and add the following student data.

```
{
"students": [
{
    "id": "1",
    "name": "sai",
    "course": "CSM",
    "Marks": 90
},
{
    "id": "2",
    "name": "Vishya",
    "course": "CSM",
    "Marks": 90
},
{
    "id": 3,
    "name": "Sravs",
    "course": "CSM",
    "Marks": 90
}
```

Step-3 :- Start the JSON server

for the current JSON server version,

Execute: npx json-server db.json

The server start at

http://localhost:3000

The student records can be accessed using.

https://localhost:3000/students.

2. Implementation of Http Methods.

a) GET Method - The GET method is used to retrieve student records from JSON server

Code :-

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
        console.log(data);
})
.catch(error => {
        console.log(error);
});
```

B. Post method

It is used to insert a new student record into the server.

Code:

```
fetch ('http://localhost:3000/students',
{
  method: 'POST',
  header: {
    'Content-Type': 'application/JSON'
  },
  body: JSON.stringify({
    name: 'sai',
    Course: 'ESE',
    Marks: 92
  })
})
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {console.log(error);
});
```

C. PATCH method:-

Code:

```
fetch ('http://localhost:3000/student/record
2', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    Marks: 96
  })
})
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

D. PUT Method:

```
fetch ('http://localhost:3000/students/1',{
  method : 'PUT,
  headers: {
  'content-type': 'application/json'
 }
body : JSON.stringify [{
       name :'Sai"
       course : 'AI/ML',
        Marks : 95
  })
})
. then (response => response.json())
- then (data => {
       console.log (data);
  })
- Catch (error => {
       Console.log (error);
  });
```

E. DELETE Method

code:

```
fetch ("https://localhost:3000/students/
  Method : 'DELETE'                    3",{
})
 .Then (response => {
    if (response.ok){
       Console.log ("Student deleted
                          Successfully");
 }
})
 .catch (error => {
        Console.log (error)
 });
```

4. Develop CRUD operations using MongoDB.

Aim:- To develop a web application using Node.JS, Express.JS and MongoDB to Perform CRUD operations.

1. Installation and project setup.

```
mkdir MongoDB CRUD
Cd MongoDB-CRUD
npm init -y
npm install express mongoose.
```

Program:- Server.JS

```
Const express = require ('Express');
Const Mongoose = require ('mongoose');
Const app = express();
Const PORT = 3000;
app.use (express.json());
Mongoose.connect ("mongoosee://
    127.0.0.1:  270>/college)
.then (() => console.log ("MongoDB
    Connected"))
.catch ((err => console.log (err));
```

```
# Create Student
Const Student.Schema = new
Mongoose.schema({
        name: {
            type: String,
            required: true
        },
        Course: {
            type: String,
            required: true
        },
        Marks: {
            type: Number,
            required: true
        }
    });

# Create Student Model
Const Student:
    Mongoose.Model('Student',
                   studentschema);

// Create add
app.post('/students', async(req, res)=>
  { try {
    Const Student = new Student(req.
                                body),
Const SavedStudent = await Student.save();
 res.status(201).json(savedStudent)
}
catch(error){
    res.status(400).json({message:
                          error: message});
  }
});

// Read: get all students
app.get('/students', async(req, res)=>{
  try {
    Const student: await
     student.find();
     res.status(500).json({Message:
     error message});
  }
});

//Read: get student by ID
app.get('/student/:id', async(req, res)
 =>{ try {
     Const Student = await
     student.findById(req.params.id)
```

```
if (!student){
    return res.status(404).json({
message = "student Not found"
});
res.status(200).json(student)
}
catch (error){
  res.status(400).json({message:
                    error.message});
  }
});

//update
app.put("/students/:id", async
                    (req,res) => {
  try{
    const student = await
    student findBy ID'and Update(
      req.params.id
        req.body
    {
      new:true, true})
);
```

```
if(!student){
  return res.status(404).json({
    message: "student Not found"
  });
}
res.status(200).json(student);
}
catch(error){
    res.status(400).JSON({message:
                    error.message});
  }
});

//Delete
app delete("/students/:id", async(req,res)
  =>{
  try{
  const student_await student,
    findBYIdANdDelete(
      req.params.id
  );
if(!student){
    return res.status(now.Json({
      message: 'student not found'
  });
}
```